Sundarban: Technical Documentation

Team: Sundarban · **Competition:** FinTechathon 2026, International Track, Topic F (Agent Wallet)
Repository: https://github.com/Bijoy2406/Sundarban · **License:** MIT

> **Safety boundary:** test tokens only. Local Anvil (chain id 31337) and Ethereum Sepolia (11155111). Never mainnet, never a wallet that holds real funds.

This document covers what the organizers ask for: **system architecture** (section 2), **core algorithm** (section 3) and **security design** (section 6, with the full assessment in security_self_assessment.md). Sections 4, 5 and 7 to 10 describe the components, the evidence and how to run it. The chain layer has its own chapter: tech_chain_layer.md.

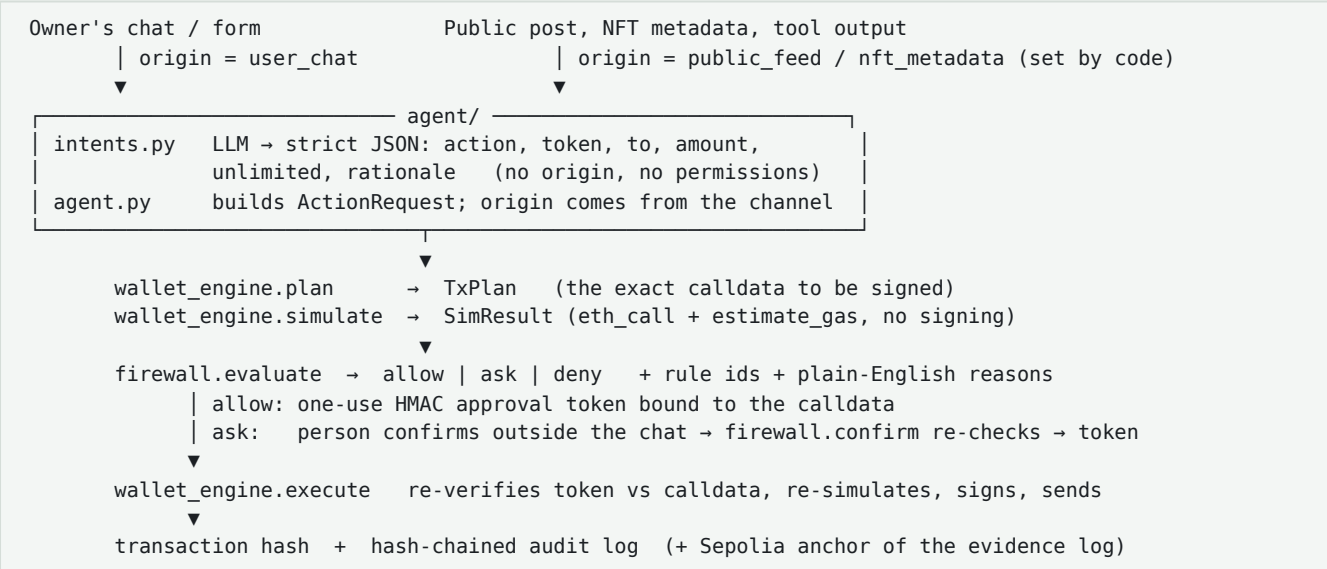
1. Problem and design goal

An AI agent that can move money is exposed to text it did not write: chat messages, public posts, NFT metadata, tool output. In May 2026 an encoded public reply plus a gifted NFT made an AI agent's wallet send about $150,000 in tokens. No key was stolen; the agent trusted text it should not have.

Sundarban is built on one rule: **permission never comes from text.** The language model only extracts a *proposed* action. A deterministic Python firewall (no AI) decides *allow*, *ask* (a person must confirm) or *deny*. The only code that can sign a transaction refuses to act without a single-use approval token that the firewall issued for the exact bytes being signed. Every decision is written to a tamper-evident log.

The name: like the Sundarbans mangrove forest absorbs the storm before it reaches the land, the firewall absorbs the attack before it reaches the funds.

2. System architecture



Layer	Folder	Responsibility
Shared contract	`shared/types.py`	`ActionRequest`, `TxPlan`, `SimResult`, `Decision`, `AgentReply`; guarded by `tests/test_contract.py` so no one changes a field or signature unnoticed

Layer	Folder	Responsibility
Chain layer	`wallet_engine/`	build, dry-run and (only with a valid token) send ERC-20 transactions; read balances and allowances
Firewall	`firewall/`	rules, runtime state, provenance, approval tokens, audit log, red-team data and runner
Agent	`agent/`	LLM client, strict intent parser, protected agent, deliberately unsafe naive agent, synthetic public feed, attack replay
API and UI	`api/`, `ui/`	FastAPI backend (port 8000) and React/Vite frontend (port 5173, proxies `/api`)
Evidence	`evidence/`	Sepolia transaction hashes, the audit log of that run and its anchor, red-team report

Backend is authoritative. The browser never supplies a decision. `/api/execute` rebuilds the plan, simulates it and calls the firewall on the server; an `ask` is remembered on the server and continues only through `/api/confirm` with a request id.

3. Core algorithm

3.1 The decision procedure (`firewall/policy.py:evaluate`)

```
evaluate(request, plan, simulation):
 1. SNAPSHOT   copy request, plan and simulation once, with strict type and length checks.
               Anything unusable → deny (DEF-01). Every later rule reads the same copy.
 2. RULES      load rules.yaml with the strict loader (unknown or duplicate keys, bad numbers →
               unusable → deny).
 3. FINDINGS   run EVERY check and collect (rule id, deny|ask, reason); nothing stops early:
                 origin (PRV-01) · kill switch (KIL-01) · chain (NET-01) · token contract (TOK-01)
                 calldata is plain hex and decodes (SIM-01)
                 calldata matches the request to the last digit (SIM-01)
                 dry run succeeded, outflow ≤ requested (SIM-01)
                 transfer: burn address (DST-01) · look-alike (ADR-01) · limits (LIM-01/02/03)
                           · new recipient (ALW-01)
                 approve:  unlimited (APR-01) · look-alike (ADR-01) or not allowlisted (APR-02)
                           · limits
                 too many recent actions (VEL-01)
 4. DECIDE     any deny → deny;  else any ask → ask;  else allow.   (default is deny)
 5. COUNT      an action from a trusted origin is counted for VEL-01 (outsiders cannot throttle
               the owner). The window is kept in logs/velocity.jsonl, so a restart does not hand out
               a fresh budget; an unreadable file counts as 'at the limit'.
 6. ALLOW      issue the approval token (120 s) → write the audit entry (must succeed) →
               record the spend in the HMAC-chained ledger (must succeed).  Any failure → deny.
    ASK        remember (action, token, recipient, amount) for this request id for 600 s;
               return no token.
    DENY       return the rule ids and reasons; nothing is issued.
```

The numbers a rule uses come from the **calldata** (`wallet_engine/binding.py:token_binding`), not from the request text, and must equal the request to the last digit. Money is `Decimal` or integers, never floats.

The rule table (16 rules, with thresholds and the exact function that enforces each) is section 3.2 of the security self-assessment. Active demo limits in `firewall/rules.yaml`: auto-approve 50 TUSD, maximum 500 TUSD per transaction, 1,000 TUSD per rolling 24 hours, 5 actions per 600 seconds, trusted origin `user_chat`, two allowlisted addresses.

3.2 Confirming an “ask” (`policy.confirm`)

A person answers an `ask` outside the chat (the UI Confirm button, which calls `/api/confirm`). `confirm` then:

1. re-runs the checks with the person’s acceptance of the *ask* reasons, but **every deny rule still applies** (a kill switch turned on in the meantime, for example, denies);
2. requires an open question for exactly this request id and exactly this transaction (CNF-01 otherwise);
3. answers only once, within 600 seconds;
4. then issues the token the same way as an `allow`.

3.3 The approval token (`firewall/tokens.py`)

`"<expiry>.<HMAC-SHA256>"` over `[request_id, action, token, recipient, amount, expiry]` with `APPROVAL_SECRET`. Valid 120 seconds, accepted once, refused while the kill switch is on. Spent signatures are appended to `logs/used_tokens.jsonl` and flushed to disk before the token counts as used, so a restart does not forget them; an unreadable file refuses every token (fail closed). Because the fields are the ones read from the calldata, a token for “approve 20 to A” cannot authorise “send 500 to B”.

3.4 Execution (`wallet_engine/execute.py`)

Refuses (nothing sent) unless: the decision is `allow` with a token; `verify_token` accepts it for the fields read from the bytes about to be signed; the plan is for this wallet and the connected chain; value is 0; gas is between 21,000 and 500,000; and a fresh simulation succeeds. It then signs (EIP-1559 fees when available, gas +20% capped), sends under a lock so two requests never share a nonce, and waits up to 120 seconds for the receipt. A transaction that was broadcast but has no receipt (or whose send call timed out or dropped) is reported as **pending** with its hash, never as failed, so it is not sent twice. If the node *answers* with a JSON-RPC rejection (nonce too low, underpriced, insufficient funds) nothing is in the mempool, so that is a refusal. Events (`EXECUTE_REFUSED`, `TX_SENT`, `TX_PENDING`, `TX_FAILED`) go to the audit log on a best-effort basis and a failed write is logged as a warning, never silently; the firewall’s own decision entry is the strict one (no audit entry, no money).

3.5 Preview has no side effects (`policy.preview`)

The Preview screen must not change later decisions. `preview` runs the same rules as `evaluate` but does not count the action, record spending, issue a token, open a question or write to the audit log. `tests/test_preview_redteam.py` checks that it leaves no trace and agrees with `evaluate` on the 53 red-team cases that contain an action (the 2 read-only questions create no request).

4. Wallet engine

Summary; the full chapter is tech_chain_layer.md.

- **Chain configuration** (`chain.py`): `ACTIVE_NETWORK` chooses local or Sepolia, the RPC URL and the signing key (`WALLET_PRIVATE_KEY` locally, `SEPOLIA_PRIVATE_KEY` on Sepolia), so a Sepolia run can never use the local key.
- **ERC-20 operations** (`erc20.py`): `transfer`, `approve` (limited or unlimited) and `revoke` (approve of 0); amounts use `Decimal` with the token’s real decimals.
- **Simulation** (`simulate.py`): `eth_call` and `estimate_gas`; never raises, reports `ok=False` with a readable reason. It reads the outflow from the same calldata the firewall decodes, so it catches transactions that would fail or that differ from the request, not a token that moves more than its calldata says; that is why only our own token contract is allowed (TOK-01).
- **Test asset:** TestUSD, a dependency-free ERC-20 (18 decimals; 1,000,000 TUSD are minted to the deployer in the constructor; no owner and no later mint). Deployments are recorded in `evidence/deployments.json`.
- **Error text:** anything an exception puts in front of a person goes through `shared/redact.py`, which hides URLs and the values of secret settings (an RPC URL carries its API key).

5. Agent, public input, and the attack replay

- `agent/intents.py` asks the LLM for strict JSON with exactly six keys, validates every field, retries once with a generic correction, then refuses politely. The schema has no `origin` and no permission field; the parser has no `origin` parameter. Text from untrusted sources is fenced in `<untrusted>` tags (Unicode-normalised so look-alikes of the closing tag are neutralised). The model keeps no memory between messages.
- `agent/agent.py` orchestrates parse → plan → simulate → evaluate → (confirm) → execute. Its confirmation text shows the full 42-character address, because summaries show only `0x1234…abcd` and a look-alike can copy those characters.
- `agent/naive_agent.py` is intentionally unprotected (no firewall; it mints its own token) and refuses unless the node is chain 31337. `wallet_engine/attacks/drainer.py` plays the attacker and requires `ACTIVE_NETWORK=local` and chain 31337.
- `agent/feed.py` provides a synthetic feed of harmless and attack items; the origin is derived from the item's source field in code, never from its text.
- **The attack replay** (`agent/replay.py`) runs ONE attack with ONE extracted action against both agents, so the outcome does not depend on what the model does that day. Layer 1 (informational) tries the protected agent's own fenced reading and reports whether the model refused. The "fooled model" is the worst case, the same text read without the fence: if the real model extracts an action that pays the attacker it is used (`source: llm`), otherwise a fixed script produces the action a fooled model would (`source: scripted`), and the page says which. Then the naive agent signs it and loses the test funds (via an unlimited approval and the drainer), while the protected agent is blocked with the reasons. Local Anvil only. The local chain is rolled back (Anvil snapshot and revert) after every replay, so it can be run again; if the model cannot be reached, layer 1 says "not tested", not "refused".

6. Security design

The full assessment is security_self_assessment.md: permission tiers, every rule with its threshold and enforcement point, trust boundaries, the controls below, the OWASP LLM Top 10 (2025) and Agentic Top 10 (2026) mapping, evidence, and an honest known-risk list. In short:

Design choice	Why
Deterministic firewall, default deny	A rule table can be read, tested and proved; a model cannot
Origin set by code from the channel	A message cannot grant itself trust (PRV-01)
Numbers read from calldata, token bound to calldata	"Approve A, send B" fails; a lying plan summary cannot pass
Single-use, 120-second token (keyed by signature, canonical expiry); never sent to the browser; kill switch cancels tokens	A leaked or replayed token is useless
Human confirmation for risky actions, outside the chat	An agent cannot confirm for itself
No audit entry, no money	Every `allow` has a record before it has an effect
Limits count approvals as spending	An allowance can be pulled out later
Operator key (`OPERATOR_TOKEN`, header `X-Operator-Token`) for Confirm and for switching the kill switch **off**; fail closed when unset	The chat agent, and anything that can only talk to the chat, cannot confirm or unfreeze: a second secret is needed (switching the kill switch **on** stays open)
Host allowlist (`localhost`, `127.0.0.1`)	A web page cannot call the local API through DNS rebinding
Spent tokens, the velocity window and the daily-spend ledger kept on disk	A restart does not reset the limits
Secrets redacted in errors and logs	Provider errors carry RPC URLs and API keys

Design choice	Why
Demo attacker, naive agent and replay are local-chain only	They cannot touch Sepolia

Audit log and its Sepolia anchor

`firewall/audit.py` writes JSON Lines linked by SHA-256 hashes. `python scripts/verify_audit.py` detects an edited, deleted, inserted or re-ordered line, and a damaged log is never silently extended. Appends are safe across threads and processes (a lock on `<log>.lock`: `fcntl` on Linux/macOS, `msvcrt` on Windows, an exclusive-create lock file elsewhere). A shortened log is still a valid chain, so the head hash of the evidence log is written into a Sepolia transaction (`wallet_engine/scripts/anchor_audit.py`; record `evidence/audit_anchor_sepolia.json`; transaction `0xf21f8ed7…61ccf7`). `anchor_audit.py --verify` re-checks it read-only. The anchor proves the log existed in this form at that block, not that its entries are true, and it covers only the evidence log.

7. API and user interface

Endpoint	Purpose
`GET /api/health`, `GET /api/wallet`	chain/LLM status; balances and allowances
`POST /api/plan`	**Preview**: plan + simulation + `firewall.preview`; no side effects, cannot be confirmed
`POST /api/execute`	rebuild, simulate, `firewall.evaluate` on the server; sends only on `allow`; `ask` → `awaiting_confirmation`
`POST /api/confirm`	continue a server-remembered `ask` by request id; needs the operator key
`POST /api/chat`	message → protected agent (origin `user_chat`)
`GET /api/security`, `POST /api/kill-switch`	read / set the emergency stop (switching it off needs the operator key)
`GET /api/audit`	newest audit entries and the chain check
`GET /api/feed`, `POST /api/attack-replay`	synthetic feed; run one attack against both agents (local Anvil only)
`GET /api/llm/status`, `POST /api/llm/test`	LLM configuration (the key is never returned); connection test

The UI offers chat, a transaction preview with a colour-coded decision card, a confirm action, the kill switch, an audit viewer, an Operator key field and the side-by-side attack replay. It displays decisions; it never authorises by sending one. Preview, execute and chat have no login (single-user local demo, served only to the Host names `localhost` and `127.0.0.1`); Confirm and kill-switch OFF need the operator key, which is one shared secret on the same machine (risk R-03).

8. Reproduce the local setup

Requirements: Python 3.14 (the only version tested; 3.11 to 3.13 not verified; exact package versions in `requirements.lock`), Node.js 20.19+ or 22.12+, Foundry (anvil, forge, cast), Git. The step-by-step path (setup, `.env`, four terminals, health check) is the README, sections 1 to 4: README.md. In short: `anvil`; `python wallet_engine/scripts/deploy.py`; `uvicorn api.main:app --port 8000`; `cd ui && npm run dev`; open http://localhost:5173. Set `OPERATOR_TOKEN` in `.env` (16+ random characters) and paste it into the UI's Operator key field to Confirm or switch the kill switch off. For a clean demo, reset the state files first (README, "Reset the demo state").

9. Tests and evidence

Evidence	Result	Where / command
Automated tests, local Anvil + TestUSD	1357 passed, 1 skipped	`python -m pytest -q`
Automated tests, no chain	1293 passed, 65 skipped (skipped is not passed)	`python -m pytest -q`
Red-team suite	55/55 as expected; 36 denied, 4 held, 0 of 40 attacks let through; 0/15 legitimate requests wrongly denied	`python -m firewall.redteam.run --no-write`; report `evidence/redteam_report.md`
Same attacks through both real agents	protected agent executed 0/40; naive agent 38/40	`python -m firewall.redteam.run --agents --no-write` (local Anvil)
Sepolia transactions through the full pipeline	6 sent, 2 denied	`evidence/tx_hashes.csv` (hashes and explorer links in the self-assessment, section 7.3; the wallet's complete history of 15 sends, including 4 unintended test-run transactions, is in section 7.4)
Audit log of that run	16 entries, chain intact, head anchored on Sepolia	`python scripts/verify_audit.py evidence/audit_sepolia.jsonl`

The organizer confirmed that Sepolia testnet transaction hashes are accepted as on-chain execution evidence. The test-file index with what each file proves is in the README ("Test cases").

10. Testing and known limitations

- A skipped test did not pass; it means its prerequisite (a local chain) was missing. The five end-to-end scenarios in `tests/test_e2e.py` and the thirteen in `tests/test_live_agents_api.py` count only when they run against Anvil.
- The red-team result measures the firewall with structured requests. It does not measure how often a real language model is fooled; the replay labels whether it used the real model's action or the scripted one.
- Windows audit-log locking (`msvcrt`) and the portable fallback are exercised by forcing those code paths on Linux in `tests/test_audit_locking.py`; a run on a real Windows machine is not recorded.
- Open items, each with an owner and a status, are in section 8 of the self-assessment. The main ones: the API has no login for preview, execute and chat and one shared operator key for Confirm, the firewall and the signing key share one process, `chain_id` is `null` in `rules.yaml` so NET-01 relies on the node, and deleting local state files weakens the daily cap.